

Software Programming

IGS1931

page1

Week 5 - Tools, Conditionals, Recursion & Rock Paper Scissors

Prof. Humayun Kabir, Thursday Session

Where does Python code live?

page2

	Shell / IDLE	.py file	.ipynb notebook	Google Colab
How you run it	Type a line and press Enter	Run the whole file	Run one cell at a time	Run cells in a browser
Is it saved?	No	Yes, as text	Yes, code and output	Yes, in Drive
Best for	Quick experiments	Programs and projects	Learning and homework	Shared PCs or no setup
Watch out for	Hard to rerun many lines	Output resets each run	Cells can run out of order	Sign-in and session resets

Different tools, different jobs

Same code, three places

page3

Shell / IDLE

```
>>> x = 5
>>> print(x * 2)
10
```

Each line runs immediately. Closing the window forgets x.

.py file: double.py

```
x = 5
print(x * 2)

Output: 10
```

The whole file runs from top to bottom with a fresh start.

.ipynb cell

```
x = 5
print(x * 2)

10
```

The output stays next to the cell.

Notebook trap: cells remember the order you ran them. If the result looks strange, restart the kernel and run all cells.

input() shows a text box below a notebook cell. In IDLE or the terminal, it waits on the next line.

Which tool should I use?

page4

Decision guide

- One expression or a quick test: Shell
- A program you will run again: .py file
- Coursework with notes and output: notebook
- No local setup or a shared PC: Colab

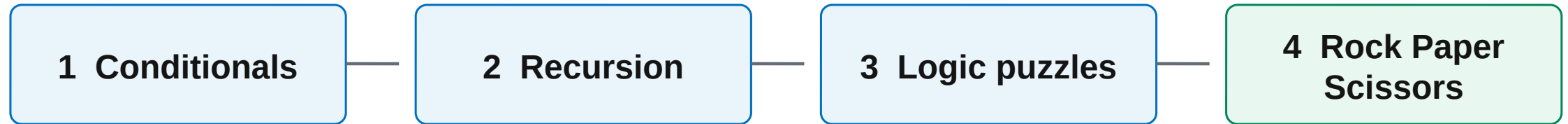
Colab on a shared PC

- Open a private browser window.
- Sign in with your student email.
- Open the Colab link from the course README.
- Use File > Save a copy in Drive.
- Sign out and close the window when done.

In this course: notebooks for lectures and homework, .py files for later projects, and the shell for quick checks.

Today's roadmap

page5



- Predict what branching code prints.
- Choose between nested if, elif, and chained comparisons.
- Find a function path that returns None.
- Separate a recursive function into a base case and a smaller call.
- Build a small interactive game.

Compound statements and indentation

page6

```
x = 3
if x > 0:                # header ends with ':'
    print('x is positive') # indented body
```

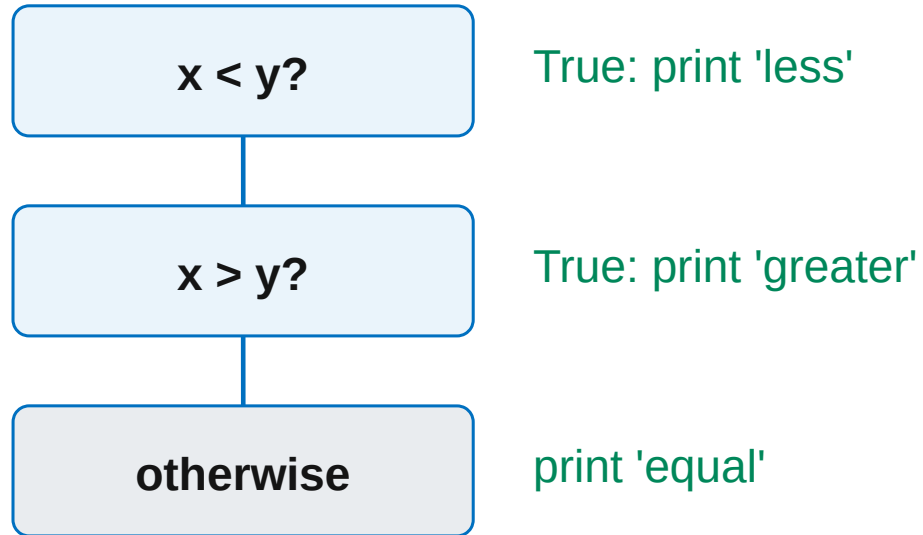
- A compound statement has a header and an indented body.
- Indentation marks the scope of def, if, else, and similar blocks.
- An unindented line after the block always runs.
- pass is a placeholder that does nothing.

```
x = -2
if x < 0:
    pass # TODO: handle
negatives
```

Indentation is part of Python's syntax.

if, elif, else: first True wins

page7



Python checks conditions from top to bottom. The first True branch runs. Python skips the rest.

```
score = 95
if score >= 50:
    grade = 'Pass'
elif score >= 90:
    grade = 'A'
```

score = 95 gets 'Pass'. The general case came first.

Nested and flat conditionals

page8

Nested

```
x, y = 3, 5
if x == y:
    print('equal')
else:
    if x < y:
        print('less')
    else:
        print('greater')
```

Flat equivalent

```
x, y = 3, 5
if x < y:
    print('less')
elif x > y:
    print('greater')
else:
    print('equal')
```

Positive single-digit number

```
x = 5
if 0 < x:
    if x < 10: ...
if 0 < x and x < 10: ...
if 0 < x < 10: ...
```

Predict the output: indentation and state

page9

What prints? 1

```
a = 5
if a > 3:
    a = a * 2
print(a)
```

Answer: 10

What prints? 2

```
a = 5
if a > 8:
    a = a * 2
else:
    a = a - 1
a = a + 1
print(a)
```

Answer: 5

What prints? 3

```
a = 5
if a > 3:
    a = a + 1
if a > 5:
    a = a + 10
a = a + 1
print(a)
```

Answer: 17

The second if sees the updated value of a.

Predict the output: missing branches

page10

Variable never created

```
score = 40
if score >= 50:
    result = 'pass'
print(result)
```

NameError: result is not defined

Wrong branch order

```
x = 95
if x >= 50:
    print('C')
elif x >= 70:
    print('B')
elif x >= 90:
    print('A')
```

Output: C

A variable created inside one branch may not exist when that branch is skipped.

The silent bug: a missing return

page11

```
def sign(n):  
    if n > 0:  
        return 'positive'  
    elif n < 0:  
        return 'negative'  
    # n == 0 has no return  
  
print(sign(5))    # positive  
print(sign(0))    # None
```

A function does not crash when a path reaches the end. It returns None.

Test boundary values. Here, 0 is the missing case.

Fix: make every path return a value.

Conditional debugging checklist

page12

#	Check	Example	Symptom
1	= or ==	if n = 0:	SyntaxError
2	Branch order	>= 50 before >= 90	Specific case never runs
3	or or and	m >= 1 or m <= 12	True for every number
4	Boundary	age > 18	Rejects exactly 18
5	Variable setup	value set in one branch	NameError or UnboundLocalError

Run one debug cell at a time. Name the checklist item before you fix it.

How to attack logic puzzles

page13

Five steps

- List every case.
- Mark the boundary values.
- Put the specific case first, or make cases exclusive.
- Add your own edge-case tests.
- Move repeated logic into a helper function.

FizzBuzz-Bazz

3: Fizz 5: Buzz 7: Bazz
35: BuzzBazz
105: FizzBuzzBazz

Why separate if
statements?

Weird or Not

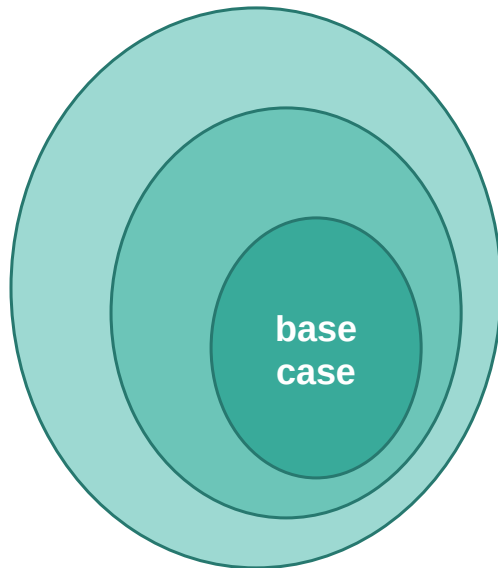
Odd: Weird
Even 2-5: Not Weird
Even 6-20: Weird
Even above 20: Not Weird

More notebook problems: game rules, median of three,
queens, days in a month, tax, valid date, next day.

What is recursion?

page14

Recursion solves a problem by calling the same function with a smaller version of the problem.



```
def count_down(n):  
    if n == 0:  
        return  
    print(n)  
    count_down(n - 1)  
  
count_down(3)
```

The next call must move closer to the base case.

Two parts of a recursive function

page15

Base case

A simple case solved directly. It does not call the function again.

Recursive case

A call with a smaller or simpler version of the problem.

Three questions

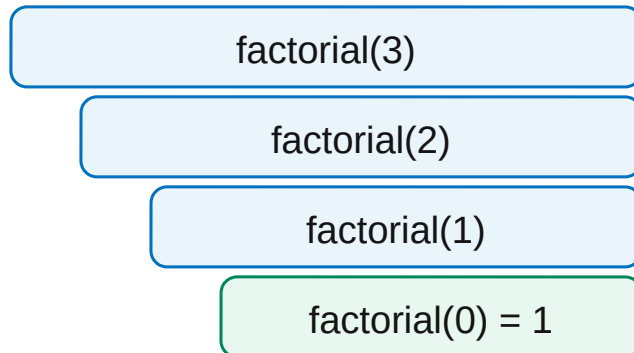
- Is the base case correct?
- Does every call move toward it?
- If the smaller call works, does the combining step give the right larger answer?

factorial(3): calls and returns

page16

```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

Calls stack up



```
factorial(3)  
= 3 * factorial(2)  
= 3 * (2 * factorial(1))  
= 3 * (2 * (1 * factorial(0)))  
= 3 * (2 * (1 * 1))  
= 3 * (2 * 1)  
= 3 * 2  
= 6
```

The calls wait on the way down. Results return on the way up.

Three recursion bugs

page17

1 No base case

```
def sum_to(n):  
    return n + sum_to(n - 1)
```

RecursionError

2 Not getting smaller

```
def factorial(n):  
    if n == 0: return 1  
    return n * factorial(n)
```

n never changes

3 Missing return

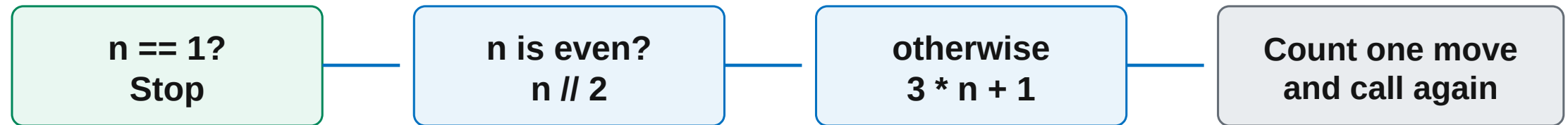
```
def power(base, exp):  
    if exp == 0: return 1  
    base * power(base, exp - 1)
```

None reaches the caller

Which recursion question fails in each example?

Conditionals and recursion: Collatz

page18



Path for n = 6

6 -> 3 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1 8 moves

- Base case: `collatz_steps(1)` returns 0.
- `collatz_steps(6)` returns 8.
- `collatz_steps(27)` returns 111.
- Other mixed challenges: `to_binary` and `count_even_digits`.

Rock Paper Scissors

page19

```
Score - You: 0 Computer: 0
rock, paper or scissors? paper
You: paper | Computer: rock
You win this round!
Score - You: 1 Computer: 0

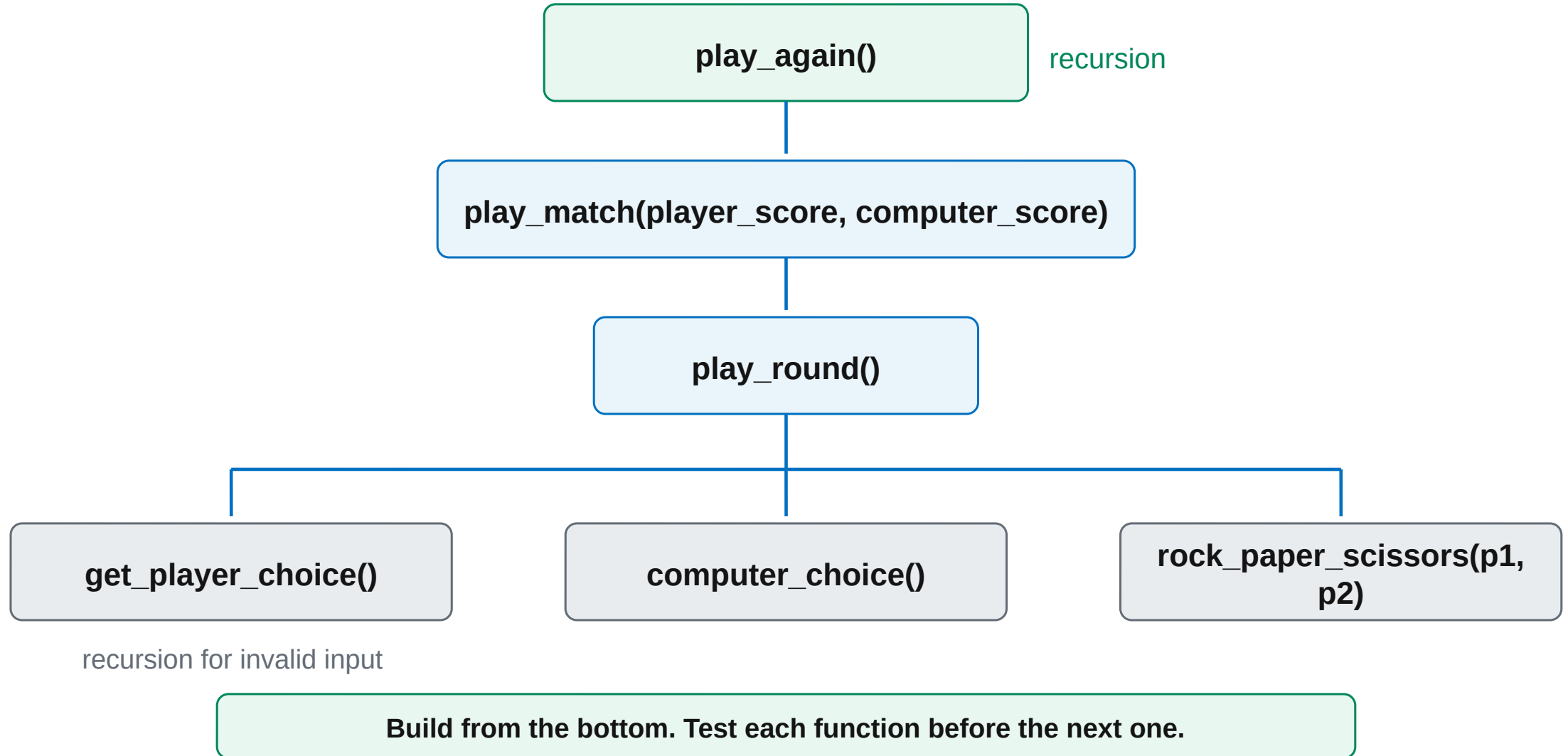
...
*** You win the match! ***
Play again? (y/n)
```

Tool	What it does	Example
<code>random.randint(a, b)</code>	Random integer, including a and b	1, 2, or 3
<code>input(prompt)</code>	Waits for text and returns a string	<code>input('Name? ')</code>
<code>strip()</code> and <code>lower()</code>	Cleans spaces and letter case	<code>' ROCK '</code> -> <code>'rock'</code>

Rules: rock beats scissors, scissors beats paper, paper beats rock. Same move means a draw. First to 2 wins the match.

Game plan: small functions

page20



Live coding: steps 1 to 3

page21

1

`computer_choice()`

`random.randint(1, 3)` gives a number. Use `if`, `elif`, `else` to return a move.

Test: run it four times.

2

`get_player_choice()`

Clean input with `strip().lower()`. Return a valid move. Otherwise print an error and call the function again.

Test: ' ROCK ' and 'banana'.

3

`rock_paper_scissors(p1, p2)`

Reuse the notebook problem. `p1` is the player and `p2` is the computer.

Test: win, loss, and draw.

Live coding: one round and a match

page22

Step 4: play_round()

- Get both moves.
- Print the two choices.
- Decide and print the result.
- Return the result string.

Step 5: play_match()

Situation	Action
player_score == 2	Print that the player won
computer_score == 2	Print that the computer won
Player wins round	Call again with player_score + 1
Computer wins round	Call again with computer_score + 1
Draw	Call again with the same scores

```
(0,0) -> (1,0) -> (1,0) -> (1,1) -> (2,1) -> player wins
```

Play again and bonus ideas

page23

Step 6: play_again()

- Ask: Play again? (y/n)
- If the answer is y, start play_match(0, 0).
- After the match, ask again.
- For any other answer, print Thanks for playing!

Bonus ideas

- Accept r, p, and s.
- Add a wins_needed parameter.
- Change messages based on the score.
- Add Lizard and Spock.
- Add CHEAT as a hidden input.

Fast finishers: choose one bonus and show it at the end.

What we practiced

page24

- Tracing branches line by line. Indentation sets the scope.
- Flattening nested conditions with elif and chained comparisons.
- Finding missing returns, wrong branch order, and incorrect and/or logic.
- Writing recursion with a base case and a smaller next call.
- Building a game with random input, functions, and recursion.

Shell: quick tests .py: programs Notebook: learning Colab: browser practice

Homework and next class

page25

Homework 5: Conditionals and Recursion

- Part A: three debugging questions.
- Part B: seven programming questions.
- Questions 8 to 10 must use recursion.
- Question 10 is the hard challenge.
- Complete the work individually and bring questions to class.

Submit:

Next up:

Colab: save a copy in Drive before editing.